SWE5001 PRACTICE PROJECT · TEAM 8

BLOCBUY

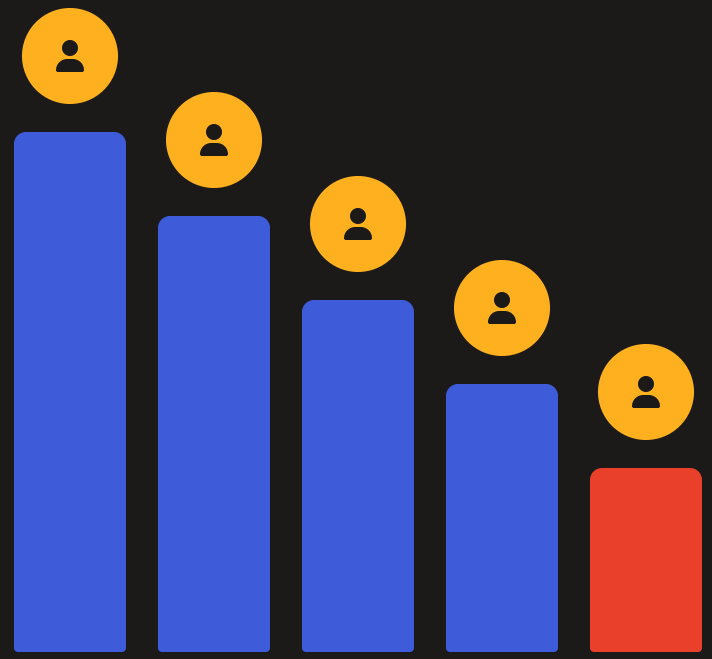
ONE FOR ALL • ALL FOR ONE

A scalable community group-buying platform

Three services • event-driven • rebuilt on every release

Lim Jiechao · Liu Jin · Panpakorn Siripanich · Wang Ziang · Yu Qianshuo

Graduate Certificate in Architecting Scalable Systems · 21 October 2026



The price drops as each member joins

Group buying only works if enough people join in time

Today: chat groups and spreadsheets

- Buyers can't see deadlines or progress
- Organisers collect money and refund by hand
- Sellers have no place for team prices

With BlocBuy

- Pay to join a team
- More members, lower price
- Team fills, or everyone is refunded automatically

JC Lim

A two-sided platform built around one interaction: forming a team

Seed a few curated launch offers

Network effect each new member lowers the price

Producers

- Sellers and admins publish offers
- Buyers who start or share a team

CORE INTERACTION

Form a team at a team price

Consumers

- Browse offers
- Join a team
- Track orders

Consumers become producers when they start or share a team

Shared, reusable services

Identity | Notifications | Payments

Future seeds: event tickets · bulk subscriptions · neighbourhood produce boxes

Scope: the full buying journey, on three services

Identity service

- Sign in with Google
- Buyer and admin roles

Commerce service

- Browse team offers
- Start or join a team, and pay
- Team succeeds, or expires with refunds
- Share invites, order history
- Admin listings

Notification service

- Team succeeded or expired
- Payment received

Not in scope: multi-region · real payments (Stripe test mode) · email or push · splitting payments into its own service (our documented next step)

Status: what is verified on AWS, and what is still open

Verified on AWS · 17 Oct

- 3 services built from nothing
  tasks 2/2 · 1/1 · 1/1, all healthy
- 9/9 smoke checks through CloudFront
- SNS → SQS routing confirmed
  with test events
- Teardown: 115 resources in 709 s
  nothing left behind
- One task: 200 req/s, 0 errors
  p95 < 60 ms (16 Oct)

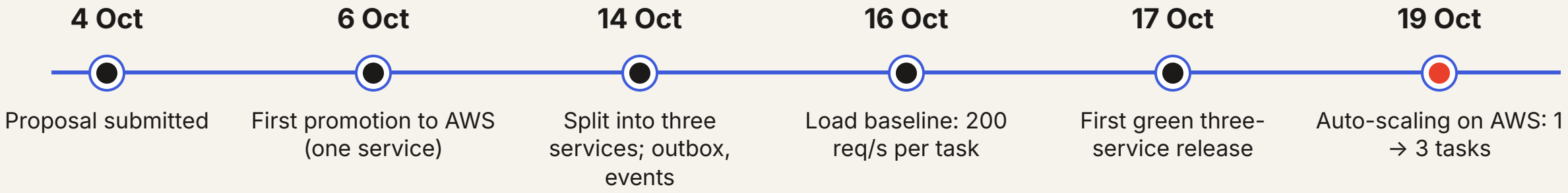
Scaling and failure tests

- Auto-scaling under load
  commerce 1 → 3 tasks on its own; ≈ 455 req/s, server p95 < 50 ms
- Queue scaling
  not run on AWS · local test passed
- Self-healing: stop a task
  not run on AWS
- Bad message to dead-letter queue
  not run on AWS · local test passed
- Identity outage drill
  not run on AWS · local test passed
- Buyer journey end to end
  not run on AWS

Still open

- An interrupted start-up can leave auto-scaling paused
  repaired by hand; no fix yet
- Database and cache in one zone
  a cost choice; multi-zone for production
- Alarm e-mails to confirm before the final release

On schedule, about 50 person-days across five people



Effort by member (person-days)

Liu Jin	**12.5**	AWS stacks, pipeline, service split
Panpakorn (Pan)	**11.5**	Messaging, scaling, login, load tests
Yu Qianshuo (Yorik)	**9**	Event outbox and relay, failure drills
Lim Jiechao	**8.5**	Lead; AWS accounts and guardrails; architecture study and proof of concept; presentation
Wang Ziang (Eric)	**8.5**	Notification consumer, dead-letter redrive, demo lead

Total ≈ 50 person-days, estimated from the issues each person delivered

WZ Eric

Why three services, not five

AD-02

One payment = one database transaction

Payment → Order → Stock → Team

Splitting this needs sagas, so it stays together in **Commerce service**

Identity: used through one small interface

Identity service

Notifications: nothing calls them

Notification service

Our rule

Split only where no transaction crosses

✕ **Rejected: five services**
Sagas everywhere, about 50–55 person-days: our whole budget

Four decisions shape the logical design

AD-02

Split by transaction boundary

No distributed transactions to build or debug

AD-04

Events through an outbox and queues

An event is sent only if the change really committed

AD-05

One shared login session

Any service checks roles without calling identity

AD-06

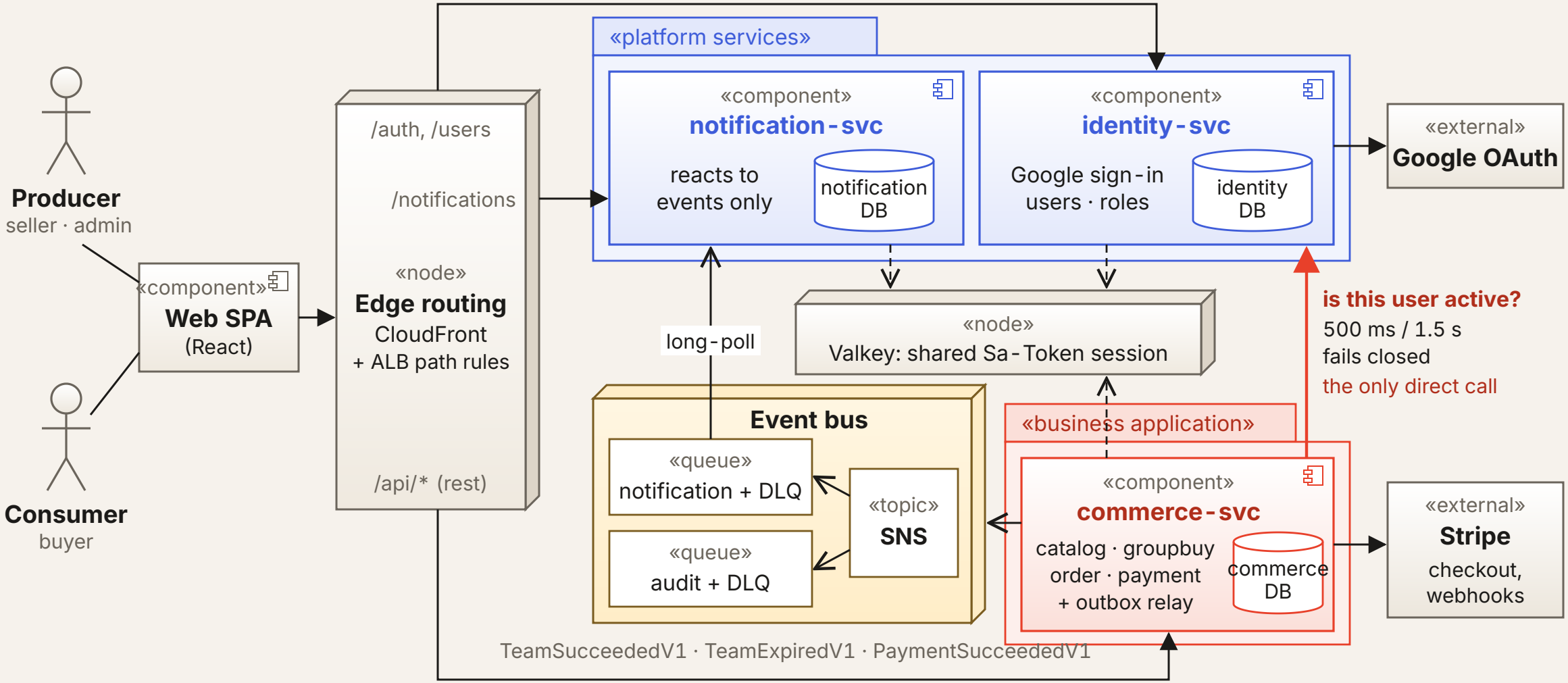
Fail safe when identity is down

Purchases are refused, never half-done

Cost of these choices: notifications lag about 2 s; purchases pause while identity is down

WZ Eric

Logical view: one direct call, everything else by events



One direct call (commerce → identity); everything else by events.

WZ Eric

Bounded contexts decide the services; events are the contract

Commerce service

Catalog
products, team prices

Group buying
teams, members

Orders
orders

Payments
payments, refunds

Identity service

Identity
users, roles

Notification service

Notification
notifications

Domain event → integration event

TeamSucceeded	TeamSucceededV1
TeamExpired	TeamExpiredV1
PaymentSucceeded	PaymentSucceededV1

- Inside a service: free to change
- Between services: versioned, IDs only

WZ Eric

How producers and consumers use the platform, and what reuse costs

Producers

- Sellers and admins publish listings
- Buyers start and share teams

Identity
one account for both sides

Commerce
offers, teams, payments

Events
any new consumer just subscribes

Notification
tells people what happened

Consumers

- Browse and join
- Track orders
- Get notified

Measured reuse

Added to the platform	What it took	Commerce edits
Audit queue	2 lines of Terraform (a name and a filter)	0
A new service	1 entry in the services map (golden-path module)	0

One public contract

`contracts/openapi.json` → generated TypeScript client
Events versioned (V1), IDs only

Every new service starts with

Login & roles · Error format · Tracing · Outbox · Duplicate protection

LJ Liu Jin

Physical decisions: managed, disposable, low cost

Fargate containers on ARM

No servers to manage · ~20% cheaper · a Kubernetes control plane alone ≈ US$73 a month

AD-07

Disposable environment

Built per session or release, then destroyed

AD-12

One database server

A database per service: isolation at low cost; its own server is the scale-out path

AD-11

Small NAT instance

≈ US$4 instead of US$43 a month

AD-10

No stored cloud keys

Short-lived tokens · secrets in Parameter Store

AD-14

MEASURED, NOT ESTIMATED

US$0.49

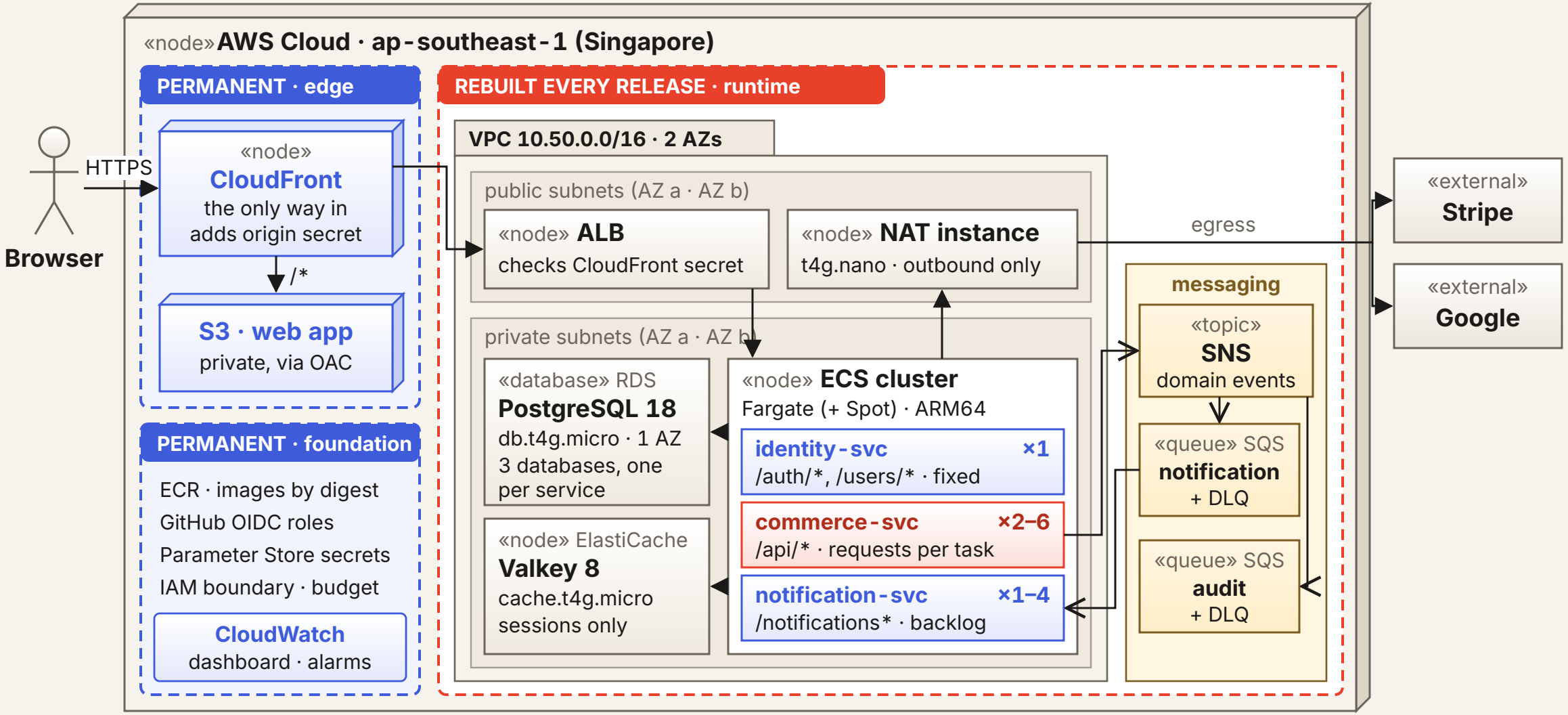
in total for the runtime's database, cache and containers across **7 up/down sessions**, 15–17 Oct

AWS Cost Explorer, filtered to the runtime services

Trade-off: less redundancy (one NAT, single-zone database), acceptable for this project

LJ Liu Jin

Physical view: private services behind CloudFront, across two zones



LJ Liu Jin

Each service scales on its own signal, and how far it goes

Commerce
Requests per task (target 12,000/min) → **2 to 6 tasks**

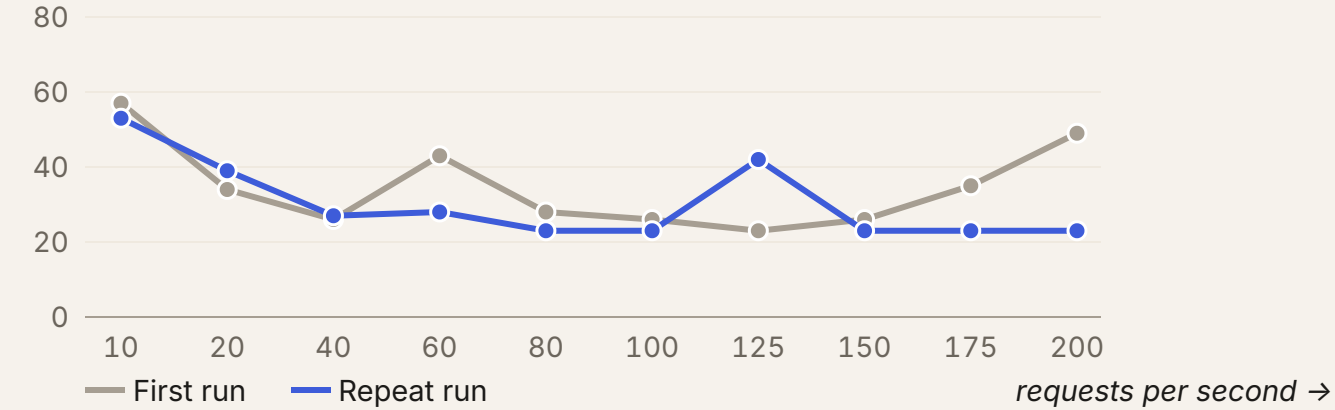
Notification
Queue backlog per task (target 50) → **1 to 4 tasks**

Identity
Fixed at **1 task**

Capacity statement

Measured: 200 req/s on one task (16 Oct); ≈ 455 req/s on 3 tasks (19 Oct, three services)
Projected: 6 tasks × 200 ≈ **1,200 req/s** for browsing **(projection, not measured)**
Next limit: the single database, then the single outbox relay (≈ 50 events/s, calculated)

Load test, one task, 16 Oct: response time (p95, ms)



200 req/s	< 60 ms	0 errors
highest tested, not saturated	95% of responses	57,600 requests

Three services, 19 Oct
commerce 1 → 3 tasks on its own; ≈ 455 req/s, server p95 < 50 ms

LJ Liu Jin

Each service owns its data



Each service reads only its own tables

per-service roles, enforced by grants and tested

The cache holds sessions only

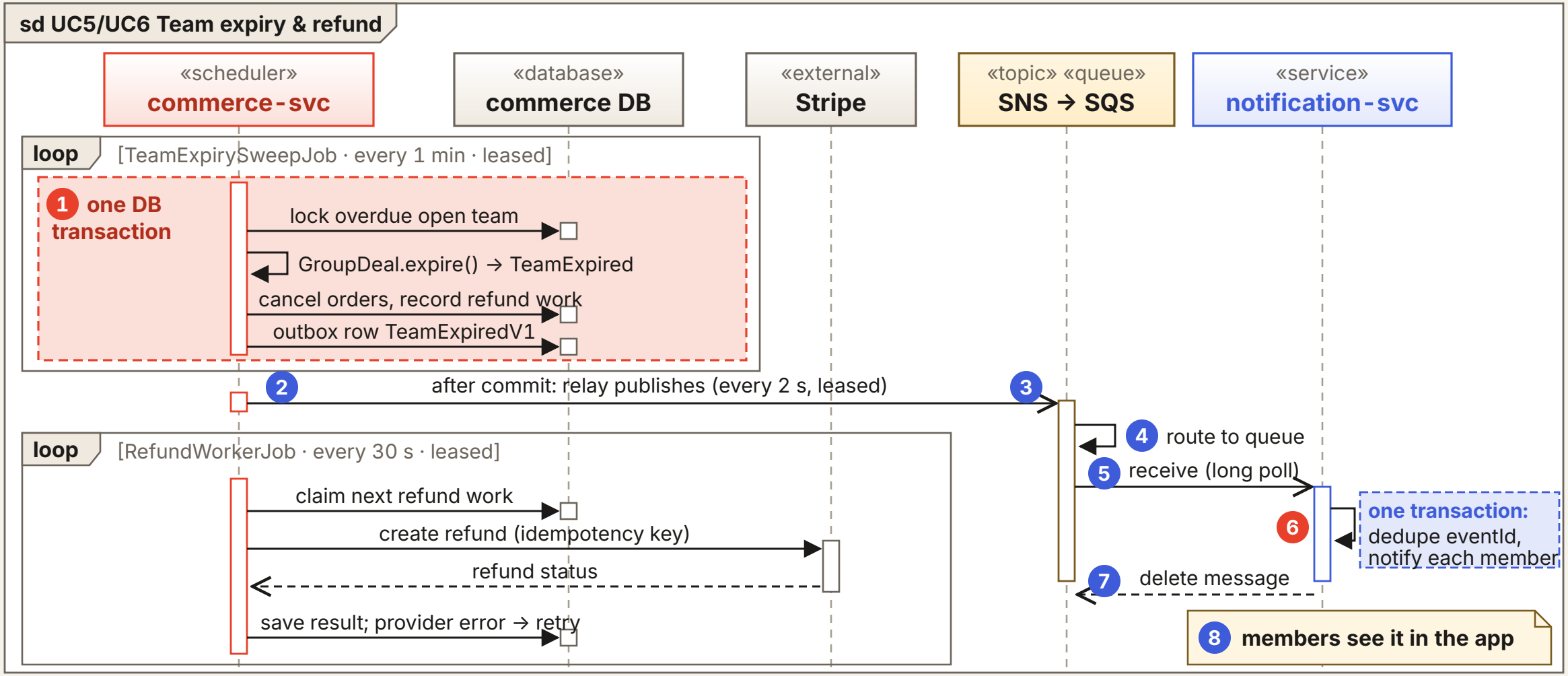
No card data stored; events carry IDs only

Concurrency

Row locks in a fixed order (**team → intent → payment**) settle the last-seat race. Scheduled jobs hold a database lease, so one task runs them however many are up.

YQ Yorik

Team expiry, end to end: one transaction in, one transaction out



YQ Yorik

Failures are expected, and handled



If...

A message arrives twice

Ignored: its (event ID, consumer) is already recorded



If...

A message keeps failing

Moved to a dead-letter queue after 5 tries; an alarm fires



If...

Publishing fails

The event stays in the outbox and is retried



No lost events

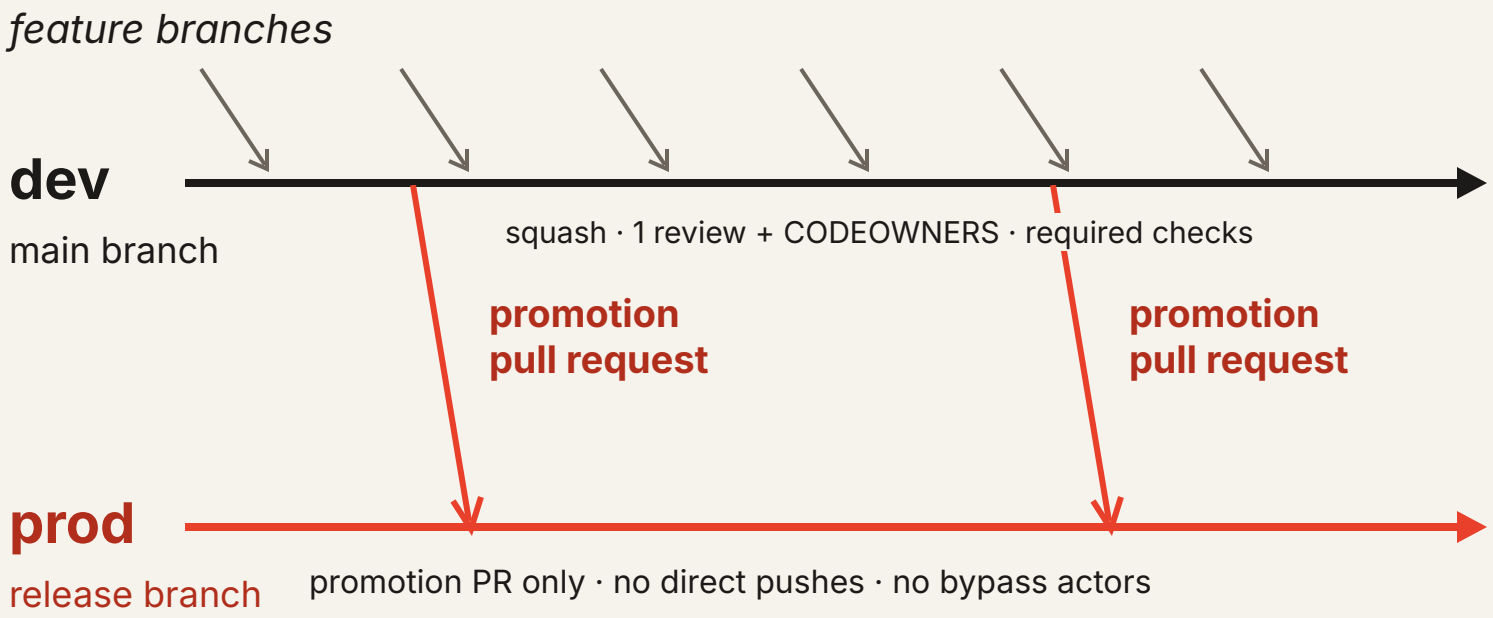


No double notifications



Bad messages isolated

One main branch; production changes only by promotion



Review + automated checks

No direct pushes to prod

Artefacts

Container images
one ARM64 image per service, stored by digest, not tag

Release record
which digests are live

API contract
one file in the repository

Every pull request is checked before it can merge

Build & test

- Unit tests and API contract check
- ≥ 80% coverage on new code (SonarCloud)

Containers

- Build and scan every service image
- Boot the app and check its health

Security

- Secret scanning (gitleaks)
- Dependency and image scans (Trivy)
- Static analysis (SonarCloud)

Infrastructure

- Terraform format, validate, test
- Policy checks (checkov)
- Plan posted to the PR

5,252 workflow runs, 16 Sep – 16 Oct

94% / 95% runs green, backend / frontend

8.6 / 3.1 min median run, backend / frontend

If any check fails, the pull request cannot merge

One promotion builds, deploys, tests and releases, then destroys it all

Approve
promotion pull request

Build
container images
≈ 5 min
first run 6 Oct: 2.5 min

Create + deploy
environment and smoke tests
≈ 32 min
first run 6 Oct: 13 min

Release
mark images as live
≈ 1 min
first run 6 Oct: 1 min

Destroy
and check nothing is left
≈ 13 min
first run 6 Oct: 10 min

51 min end to end, three services, 17 Oct

28 min on 6 Oct with one service · 115 resources destroyed in 709 s

Destroy always runs
even if an earlier step failed

Failed deploy rolls back
the ECS circuit breaker restores the last good version

Release digests in Parameter Store are the last-known-good set

Every release is a rebuild from zero, so every release also tests disaster recovery

What the real cloud taught us

1

Mocks passed, the real deploy failed

Missing cloud permissions only showed up in a real run

Test permissions for real

2

Events lost right after deploy

New SNS subscriptions take up to 15 minutes to route; nothing errors, messages just go nowhere

Deploy now waits for routing

3

Auto-scaling silently paused

A retried start-up reported success but left scaling suspended

Check scaling, don't assume it

No standing keys: who and what can touch AWS

People
IAM Identity Center

admins (3) · developers (2)
1-hour sessions

Machines
GitHub OIDC

plan / deploy / apply roles
no AWS keys in GitHub

Disclosed exception
2 IAM users for port forwarding (1 active key), inside the same guardrails

AWS Organization management account runs no workloads

Root SCP · Org-Baseline SCP · Region-Pin: Singapore + global

Sandbox OU SCP · Sandbox-Guardrails
no expensive services · small instance sizes only

Team 8 project account

dev environment
always-on EC2 host

prod environment
ECS, rebuilt per release

Guardrails above the account: they bind even its admins and root user

Live demo

1 **Buy as a customer, across three services**

Functionality

2 **Auto-scaling, on the dashboard**

Scalability, performance

3 **The pipeline that deployed it**

Maintainability

Failure handling: not shown live

Identity outage · dead-letter queue: tested locally · stop a server: not run · results in B12

- **Live on AWS · the scaling graphs are from our 19 October load test**

Thank you

Questions?

WHAT WE HOPE YOU REMEMBER

- Split where no transaction crosses
- Events never lost or doubled
- Fails safe when identity is down
- Rebuilt from scratch every release

1 → 3 tasks
auto-scaled at ≈ 455 req/s, 19 Oct

US$0.49
runtime database, cache and containers, 7 sessions

51 min
approved PR → released, tested, destroyed

github.com/PPsyrius/iss-groupbuy-team8 · d3lvq02o2e7tcn.cloudfront.net

JC Lim

Backup slides

B1a **Logical decisions (AD-02 to AD-06)**
Why did you choose X over Y? (logical)

B1b **Physical decisions (AD-07 to AD-14)**
Why Fargate, a NAT instance, one database server?

B2 **Known limitations and the way out**
Single points of failure, a large commerce service

B3 **Context map: where service boundaries may go**
What does DDD actually change?

B4 **Aggregates and their life cycles**
How do a team and an order change state?

B5 **Layers, and tests that keep boundaries in place**
How do you stop boundaries eroding?

B6 **Joining a team by paying, step by step**
Show me the payment path, not expiry

B7 **Technology and services**
Exact services, sizes and versions

B8 **Everything the real cloud taught us**
What else went wrong?

B9 **Infrastructure as code, and the actual bill**
Stacks, lifetimes and real cost

B10 **API access for producers and consumers**
Platform engineering: robust API access

B11 **Analytics: what the platform measures**
Analytics to improve the platform

B12 **Scale-out and resilience results**
Evidence behind slides 5, 14 and 17

B13 **Logical view, simplified**
Plainer companion to slide 9

B14 **Physical view, simplified**
Plainer companion to slide 13

B15 **Team expiry, numbered steps**
Plainer companion to slide 16

Click a row, or press B to come back here. Slide 24 stays up during questions; open a backup only when a question needs it.

WZ Eric

Logical decisions (AD-02 to AD-06)

ID	Decision	Why
AD-02	Split where no transaction crosses, not by nouns	Payment and expiry stay one transaction, so no sagas. Five services rejected (50–55 person-days)
AD-03	One database and one database role per service	No joins across services; enforced by grants and architecture tests
AD-04	Outbox → SNS → SQS with dead-letter queues; consumers skip duplicates	Events are never lost or applied twice, and there is no broker to run
AD-05	One shared login session in Valkey, roles inside it	Services check roles without calling identity on every request
AD-06	One direct call, commerce → identity, failing closed	An outage returns an error and Stripe retries; a buyer is never wrongly rejected

LJ Liu Jin

Physical decisions (AD-07 to AD-14)

ID	Decision	Why
AD-07	Fargate on ARM, not Kubernetes (EKS)	The EKS control plane (≈ US$73/month) costs more than all our compute; ARM is ~20% cheaper
AD-08	Fargate Spot above the minimum (1 : 3)	Cheap extra capacity; the minimum stays on regular Fargate
AD-09	CloudFront → load balancer with a secret header	HTTPS at the edge; the load balancer refuses everything else (403)
AD-10	Small NAT instance, not a NAT Gateway	≈ US$4 instead of ≈ US$43 a month
AD-11	One database server, three databases	Low cost; moving a database to its own server needs no code change
AD-12	Permanent edge, disposable runtime	≈ US$0.12–0.20 an hour while up, instead of ≈ US$125 a month always on
AD-13	No Kubernetes or Argo CD	They would cost 8–12 days of set-up before any feature work
AD-14	Own account in an AWS Organization	Separate blast radius and bill; short-lived logins; guardrails the account admin cannot lift

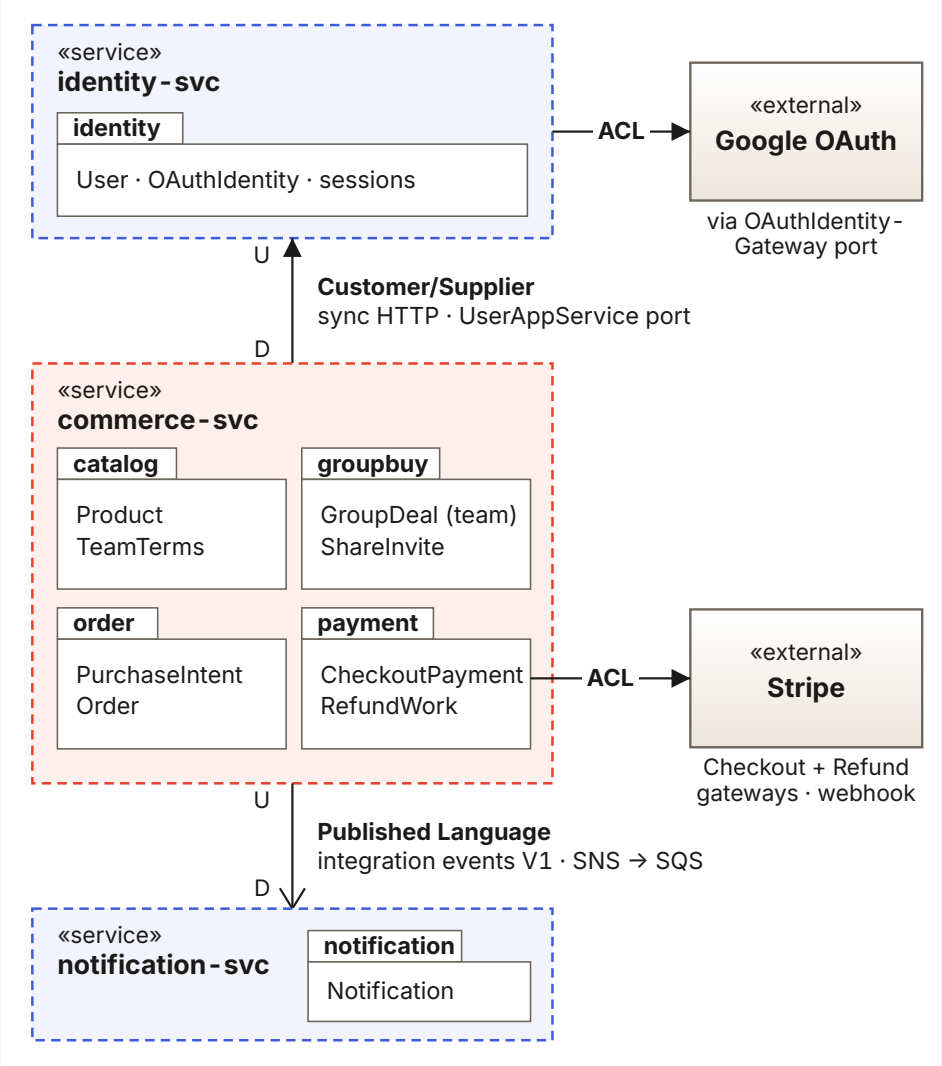
LJ Liu Jin

Known limitations and the way out

ID	Limitation	Way out
AISS-01	Database, cache and NAT each run in one zone	A cost choice; production adds multi-zone and replicas
AISS-02	The commerce service is large	Move payments out next, with a saga for refunds
AISS-03	Checkout stops while identity is down	By design (no wrong refunds); later a cache or circuit breaker
AISS-04	Internal endpoints rely on network rules only	Add a service token or mutual TLS
AISS-05	One shared database server	Move a database to its own server; no code change
AISS-06	Building the environment takes ≈ 32 min, 15 of them waiting for SNS	Start it early on the day; the edge stays permanent

WZ Eric

Context map: where service boundaries may go



- **Identity → Google:** anti-corruption layer (the OAuth gateway port)
- **Identity → commerce:** customer and supplier, one synchronous call (the user port)
- **Commerce → Stripe:** anti-corruption layer (checkout and refund gateways, webhook)
- **Commerce → notification:** published language: version 1 events over SNS → SQS

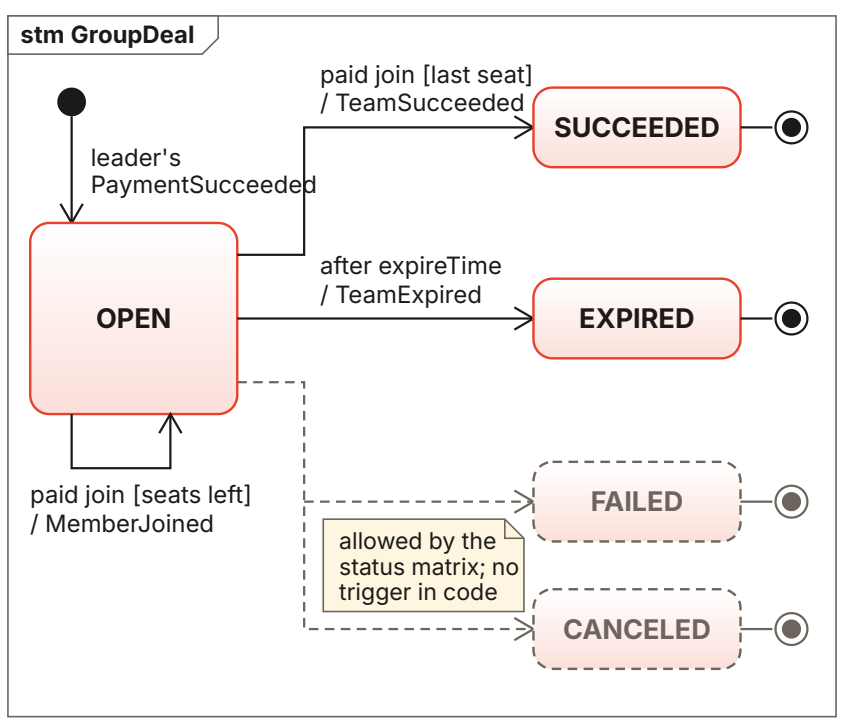
Four contexts share one service because they share transactions; the map shows where network boundaries are allowed to be.

WZ Eric

Aggregates and their life cycles

- **Team (GroupDeal):** only OPEN may change state, and only once
- **Order:** PENDING → CONFIRMED or CANCELLED
- **Purchase intent** is the order-side aggregate
- Also: checkout payment, refund work, product with team terms, notification
- Domain events: MemberJoined, TeamSucceeded, TeamExpired, PaymentSucceeded
- In the same transaction, the outbox writer turns them into versioned integration events

Be ready to explain: FAILED and CANCELED are allowed by the status table, but nothing in the code triggers them yet.



WZ Eric

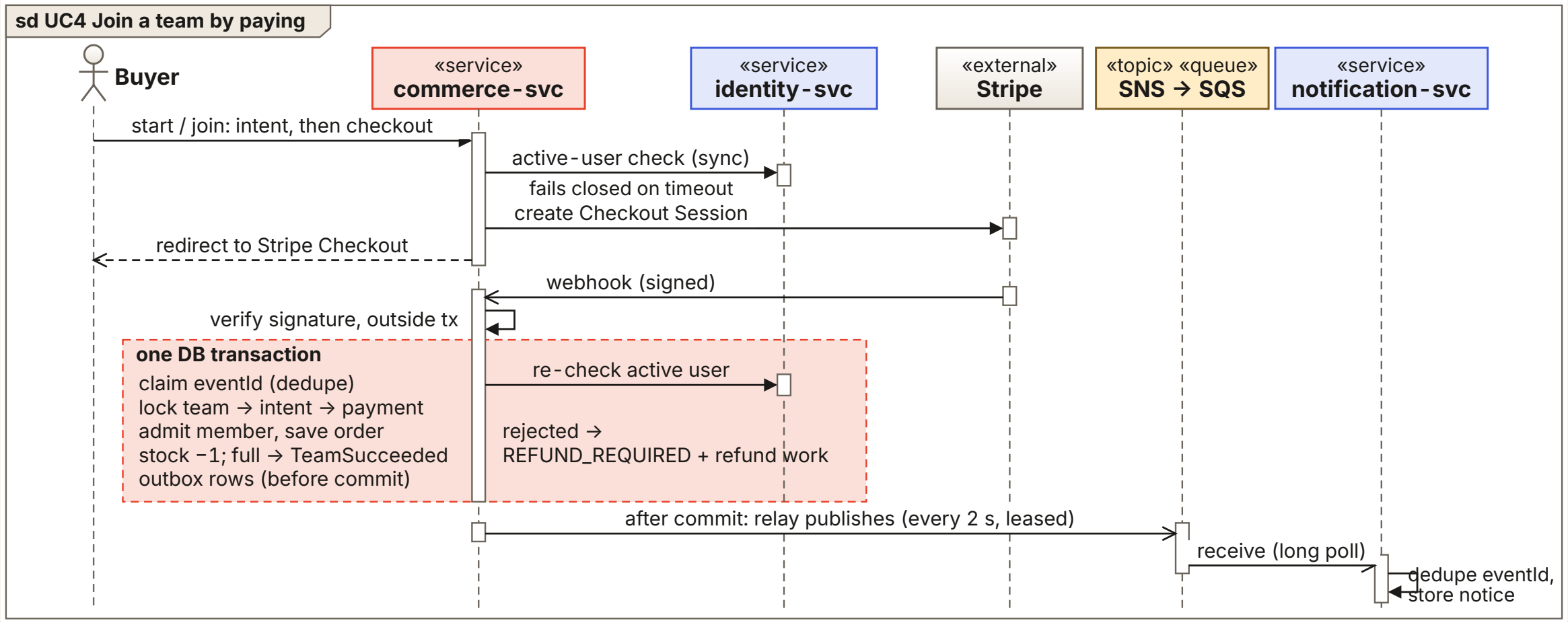
Layers, and tests that keep boundaries in place

Layer	Module	Contains
Client	`blocbuy-client`	DTOs and the integration event contracts (TeamSucceededV1, TeamExpiredV1, PaymentSucceededV1)
Adapter	`blocbuy-adapter`	REST controllers, the result envelope, the global error handler
Application	`blocbuy-application`	Commands, queries, application services, outbox relay
Domain	`blocbuy-domain`	Aggregates, domain events, ports (no framework code)
Infrastructure	`blocbuy-infrastructure`	MyBatis, Stripe, Redis, SNS/SQS transports
Assemblies	`services/*-svc`	Thin Spring Boot entry points that pick which contexts to run

Boundaries are tested: 18 architecture rules run with the unit tests, e.g. `commerce_reaches_identity_only_through_user_port`

YQ Yorik

Joining a team by paying, step by step



The signature is checked outside the transaction; one transaction then deduplicates, locks, admits the member, saves the order, takes stock and writes the outbox, re-checking that the user is active.

LJ Liu Jin

Technology and services

Concern	Choice
Edge	CloudFront (permanent: Google sign-in and the Stripe webhook depend on its address), S3 for the web app
Compute	ECS Fargate on ARM64, 0.5 vCPU / 1 GB per task, circuit breaker with rollback, Service Connect
Scaling	commerce: requests per target, 2–6, out after 60 s / in after 180 s · notification: backlog per task (50), 1–4 · identity: 1
Data	PostgreSQL 18 on RDS db.t4g.micro (TLS) · Valkey 8 on ElastiCache cache.t4g.micro (TLS and password)
Messaging	SNS (encrypted) → SQS (60 s visibility) with dead-letter queues after 5 receives, filter policies
Secrets	Parameter Store SecureString; database passwords never stored in Terraform state
Observability	CloudWatch dashboard (edge, services, messaging), dead-letter and queue-age alarms, trace ID on every request
App stack	Java 21, Spring Boot 4, MyBatis, Sa-Token, Stripe · React 19, TanStack, HeroUI

PS Pan

Everything the real cloud taught us

What happened	What we did
The first promotion failed five times (image digests, variables, a tag policy, an early 503)	Fixed one at a time; first green run on 6 Oct
A read-only container disk crashed the web server's temp folder	Mounted a writable /tmp
The AWS tagging API still lists resources that are already deleted	The teardown check asks each service directly
Default database privileges exposed tables to every role	Grants set after each migration, and tested
The outbox relay marked unsent rows as failed after a mid-batch error	Caught in code review and fixed
The notification consumer was never switched on in AWS	Now enabled together with messaging
Setting up scaling was denied: tags need an extra permission	Permission added, limited to this project (17 Oct)
The deploy stopped twice: alarm e-mails were not yet confirmed	Testing opt-out for now; confirm the e-mails before the final release
An interrupted start-up leaves auto-scaling paused	Repaired by hand; no fix yet
An organisation policy blocked CloudFront metrics (us-east-1)	Policies consolidated with a narrow exception

PS Pan

Infrastructure as code, and the actual bill

Stack	Lifetime	Contents
foundation	permanent	state bucket, GitHub OIDC roles with a permissions boundary, ECR, Parameter Store
edge	permanent	CloudFront and the S3 web app (pointed at the new load balancer on every build)
runtime	disposable	network, ECS, load balancer, database, cache, SNS/SQS, dashboard, alarms: 113–115 resources

- `env-up.sh / env-down.sh / env-status.sh` (status checks that nothing is left)
- Up ≈ 32 min (17 Oct, twice); down ≈ 13 min (115 resources in 709 s)
- Golden-path `ecs-service` module: a new service is one map entry
- The apply role can only touch IAM under `/swe5001/`, limited by tags and names

Actual AWS bill (whole account, Cost Explorer): Sep US$78.51; 1–17 Oct ≈ US$65 (≈ US$4 a day), mostly the always-on development server. This project's runtime (database, cache, containers): **US$0.49** across 7 sessions.

WZ Eric

API access for producers and consumers

Clients
the web app; future partner apps

→

CloudFront
the only way in; adds the secret header

→

Load-balancer paths
/auth, /users, /notifications, /api/*; anything else 403

→

Services
identity · commerce · notification

Built

- Public contract `contracts/openapi.json`, committed; TypeScript client generated; contract check on every pull request
- Events versioned (V1): producers can add V2 without a joint release
- Internal endpoints: not routed, peer services only, not in the public contract (tested)

Not built (be upfront)

- API gateway, per-client keys, rate limiting
- Service-to-service token or mutual TLS (AISS-04)
- **Way forward:** API Gateway, or load-balancer and WAF rate rules behind CloudFront; a service token on internal endpoints

Analytics: what the platform measures

Operational, in use

- CloudWatch dashboard: edge, services, messaging
- The 16 Oct load results set the scaling target (12,000 requests per task per minute)
- Dead-letter and queue-age alarms

Business, in the data, not built

- Team fill rate and time to fill, per product
- Referrals from share invites (invites and credits are already stored)
- Start-to-success conversion; refund rate per product

Next step **Event bus** → **Analytics consumer** → **Reporting store**

A read-only consumer on the same event bus, the same pattern as the audit queue: no change to commerce.



CloudWatch dashboard blocbuy-swe5001, October test session

JC Lim

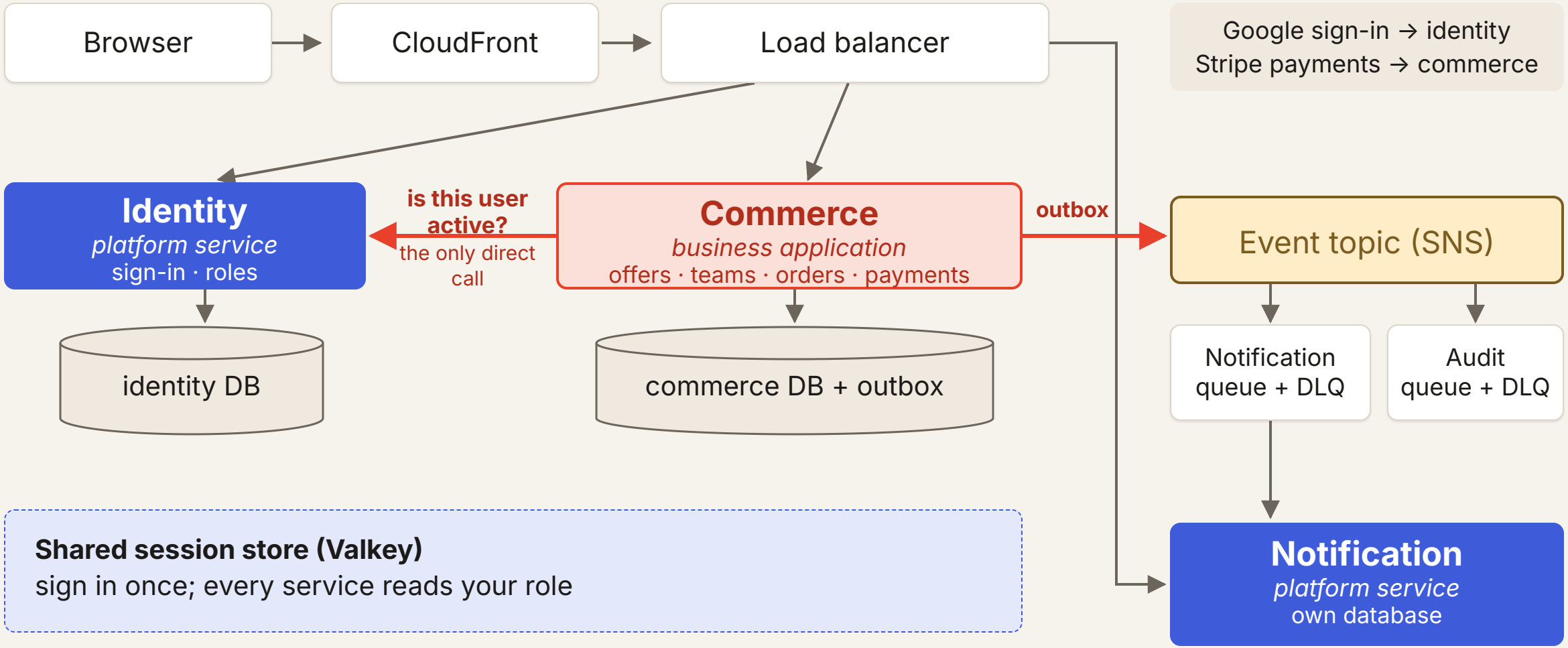
Scale-out and resilience results

Check	Target	Result on AWS
HTTP auto-scaling	≥ 80% of linear from 1 → 4 tasks; p95 < 800 ms except 3 min after a scale-out	commerce 1 → 3 tasks on its own; ≈ 455 req/s, server p95 < 50 ms. Partial: the load laptop hit its limit first, so 4 tasks and linearity are not shown yet
Queue scaling	2,000 expiries → up to 4 tasks, drained, dead-letter queue empty	not run on AWS · local test: 141 s end to end
Self-healing	Stop 1 of 2 tasks: no failed requests, replacement within 2 min	not run on AWS
Bad message	5 receives → dead-letter queue → alarm	not run on AWS · local test: passed 15 Oct
Identity outage	Browsing works; checkout returns 500 and is redelivered once	not run on AWS · local test: passed
Build from nothing	3 services stable, 9 smoke checks	17 Oct, twice: tasks 2/2 · 1/1 · 1/1; 9/9 checks; SNS → SQS verified; buyer journey end to end: not run on AWS
Teardown	Everything destroyed, absence check clean	115 resources in 709 s (17 Oct); 113 in 784 s (16 Oct)

A local result is never presented as an AWS result.

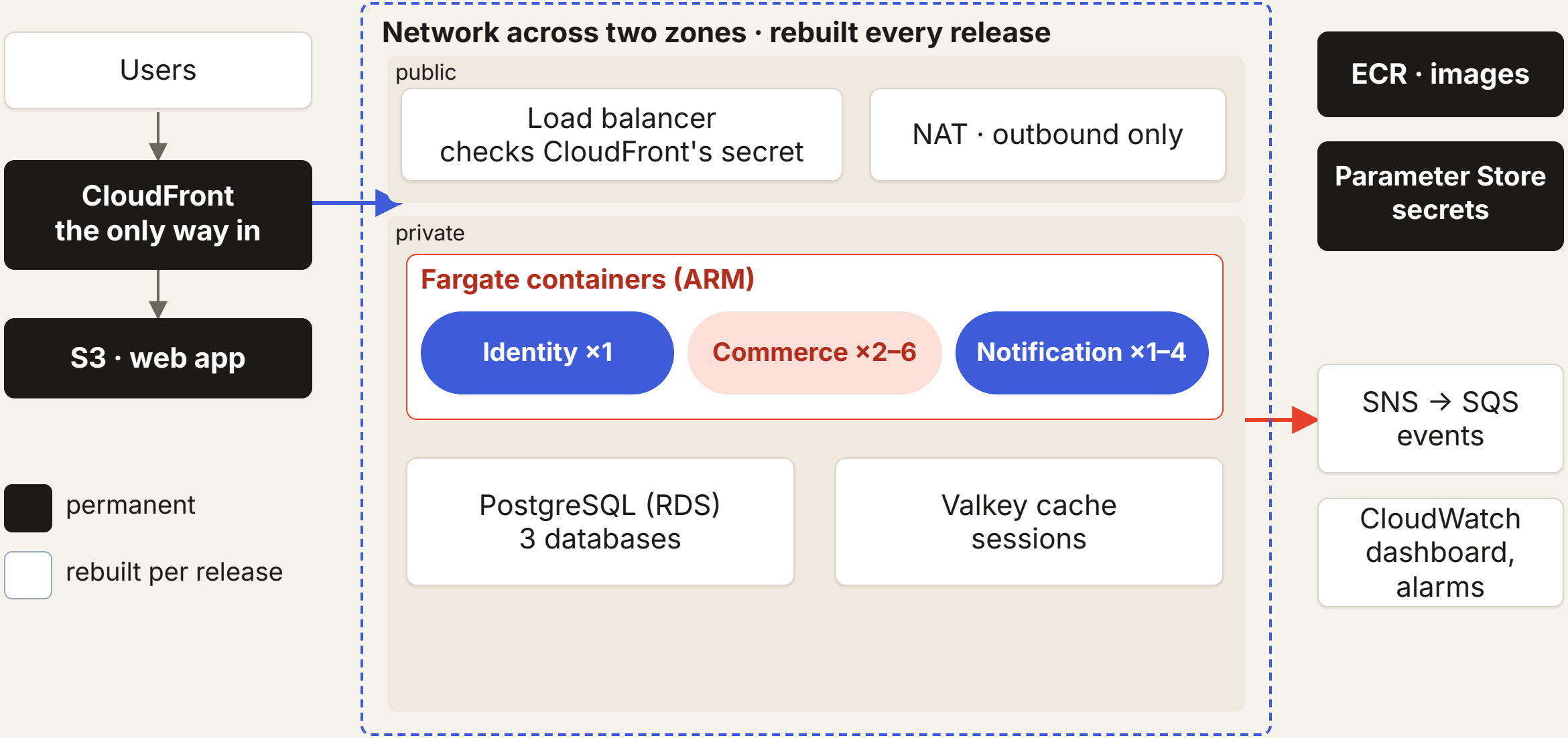
WZ Eric

Logical view, simplified



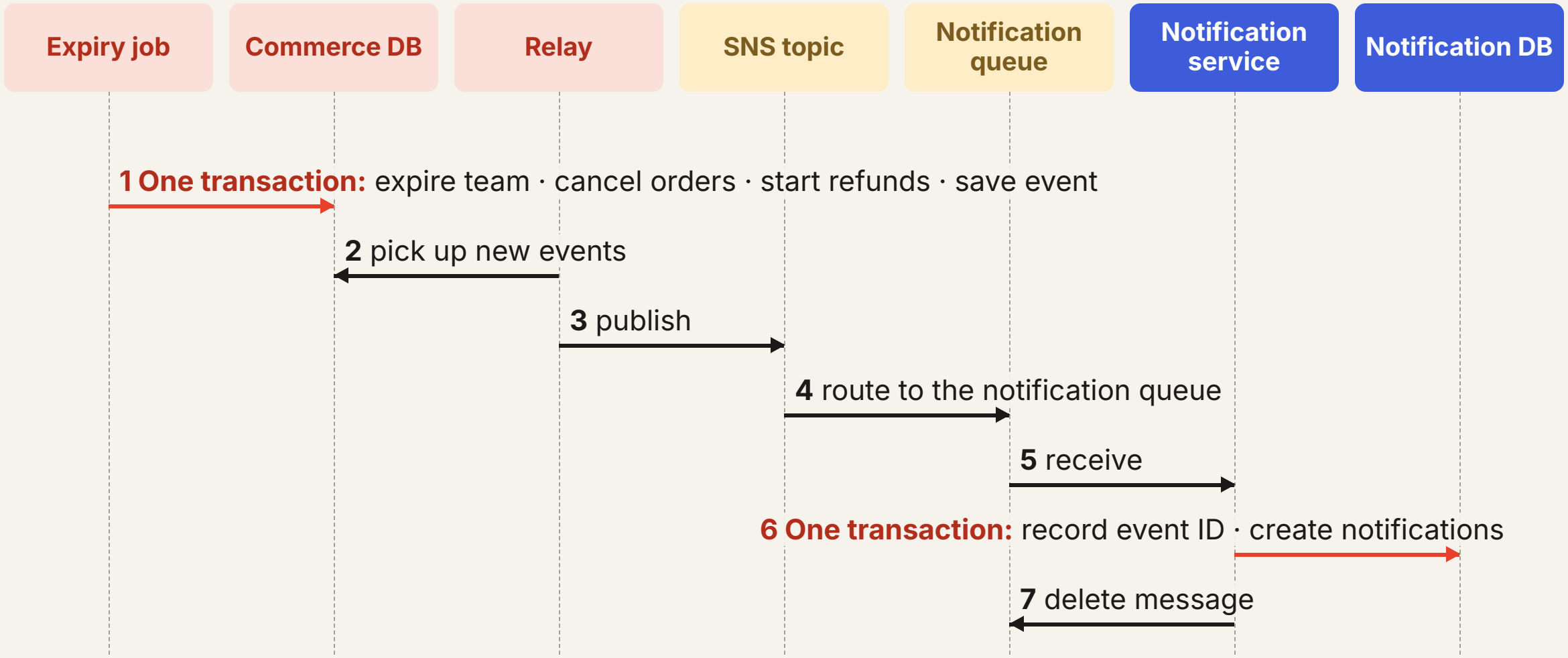
LJ Liu Jin

Physical view, simplified



YQ Yorik

Team expiry, numbered steps



8 Each member sees it in the app